Weekly Report 1 - Movie Ticket Booking System

1. Thông tin project

- Project: Movie Ticket Booking System
- Course: OOP HK1 2026-2027 Đợt 1
- Giảng viên: Huỳnh Xuân Phụng
- Tuần: 1
- Ngày báo cáo: 04/10/2026
- Thành viên: [23110062 - Nguyễn Đức Thắng & 23110066 - Nguyễn Thành Triệu]

2. Mục tiêu tuần 1

- Hiểu và giới hạn phạm vi bài toán.
- Xác định các use case chính.
- Thiết kế các class domain.
- Thiết kế cơ chế polymorphism cho seat pricing.
- Thiết kế cơ chế discount có thể mở rộng.
- Chuẩn bị skeleton Java chạy được.

3. Problem restatement

Hệ thống mô phỏng một rạp chiếu phim bán ghế cho nhiều suất chiếu trên nhiều phòng chiếu. Người dùng có thể xem phim và suất chiếu, chọn ghế, giữ ghế trong quá trình booking, áp dụng giá theo loại ghế, áp dụng discount và nhận confirmation summary.

Phiên bản đầu không bao gồm database, đăng nhập, thanh toán thật hoặc GUI.

4. Use cases

1. Browse movies. 2. Browse showtimes. 3. Select available seats. 4. Temporarily lock selected seats. 5. Apply a discount policy. 6. Confirm booking. 7. Generate booking confirmation. 8. Reject an attempt to lock an already locked or booked seat.

5. Công việc đã hoàn thành

- Tạo thư mục `final_project` và cấu trúc source code.
- Thiết kế class diagram cho Movie, Screen, Showtime, Seat và Booking.
- Tạo interface `PricingTier` và ba implementation Standard, Premium, Recliner.
- Tạo interface `DiscountPolicy` và implementation StudentDiscountPolicy.
- Thiết kế các trạng thái ghế `AVAILABLE`, `LOCKED`, `BOOKED`.
- Để `Showtime` quản lý seat-lock state.
- Tạo chương trình mẫu có happy path.
- Tạo validation scenario cho double-booking.
- Tạo README hướng dẫn compile/run.

6. Self-Explanation ban đầu

Polymorphism

Polymorphism xuất hiện tại `PricingTier`, `StandardTier`, `PremiumTier` và `ReclinerTier`, cụ thể ở method `calculatePrice(double basePrice)`. `Seat` giữ một reference kiểu `PricingTier`, nên `Booking` có thể tính giá nhiều loại ghế thông qua cùng một interface. Đây là lựa chọn phù hợp vì thêm loại ghế mới không cần sửa booking logic.

Encapsulation

Encapsulation xuất hiện tại `Showtime`, cụ thể ở các method `tryLockSeats`, `confirmBooking`, `releaseSeats` và `isSeatAvailable`. Map `seatStates` là private và chỉ `Showtime` được thay đổi trạng thái. Điều này phù hợp vì điều kiện lock/confirm cần được kiểm soát tập trung để ngăn double-booking.

Composition

Composition xuất hiện tại `Screen` chứa các `Seat`, `Showtime` tham chiếu đến `Movie` và `Screen`, còn `Booking` chứa các `Seat` đã chọn và `Showtime`. Các đối tượng này tạo thành một suất chiếu và một booking cụ thể, thay vì chỉ truyền các ID rời rạc.

7. Minh chứng chạy chương trình

Lệnh chạy:

```
javac -d out src/main/java/cinema/*.java
java -cp out cinema.Main
```

Output kỳ vọng có:

```
=== HAPPY PATH ===
Booking confirmed successfully.
Movie: Interstellar
Seats: A1, A2

=== VALIDATION: DOUBLE BOOKING ===
Rejected correctly: Seat A1 is not available.
```

Ảnh terminal hoặc output thực tế được lưu trong thư mục `evidence/` trước khi nộp.

8. Khó khăn và giới hạn

- Chưa có automated tests.
- Chưa có thời gian hết hạn cho ghế đang `LOCKED`.
- Chưa có database transaction cho nhiều server.
- Chưa có giao diện người dùng.

9. Kế hoạch tuần 2

- Bổ sung unit tests.
- Hoàn thiện pricing và discount.
- Bổ sung kiểm tra dữ liệu không hợp lệ.
- Bổ sung `GroupDiscountPolicy`.
- Cập nhật class diagram theo implementation.

10. Giải trình việc sử dụng AI

AI được dùng như công cụ hỗ trợ thảo luận và tạo bản nháp, không phải người quyết định thay nhóm. Nhóm tự chọn Java, tự giới hạn phạm vi console application, tự chốt việc để `Showtime` quản lý trạng thái ghế và tự kiểm tra source bằng `javac`/`java`.

Chi tiết từng lần trao đổi, phần AI gợi ý, phần nhóm sửa và cách kiểm chứng được ghi trong [AI Collaboration Log](ai-collaboration-log.md).

11. GitHub và lịch sử commit

- Repository public: [Dán URL GitHub thật vào đây]
- Commit hồ sơ tuần 1: [Dán commit URL thật vào đây]

Lịch sử commit và nội dung commit sẽ phân biệt rõ phần AI hỗ trợ với quyết định, chỉnh sửa và kiểm thử của sinh viên. Kế hoạch commit chi tiết nằm trong [GitHub Commit Plan](github-commit-plan.md).